

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра автоматизованих систем управління



Звіт

до лабораторної роботи №3
з дисципліни:

“Паралельні обчислення і розподілені системи”

на тему:

**“RESTful сервіси (частина 2). Розширений REST API, PUT/
DELETE, обробка винятків”**

Виконав: студент ОІ-31
Джебко Юрій

Прийняв:
Скорохода О. В.

Львів – 2026

Лабораторна робота №3

RESTful сервіси (частина 2). Розширений REST API, PUT/DELETE, обробка винятків

Мета роботи: забезпечити стійкість даних та підготувати систему до подальшої декомпозиції. Розширення REST API та інтеграція з базою даних.

Хід роботи

1.Проектування:

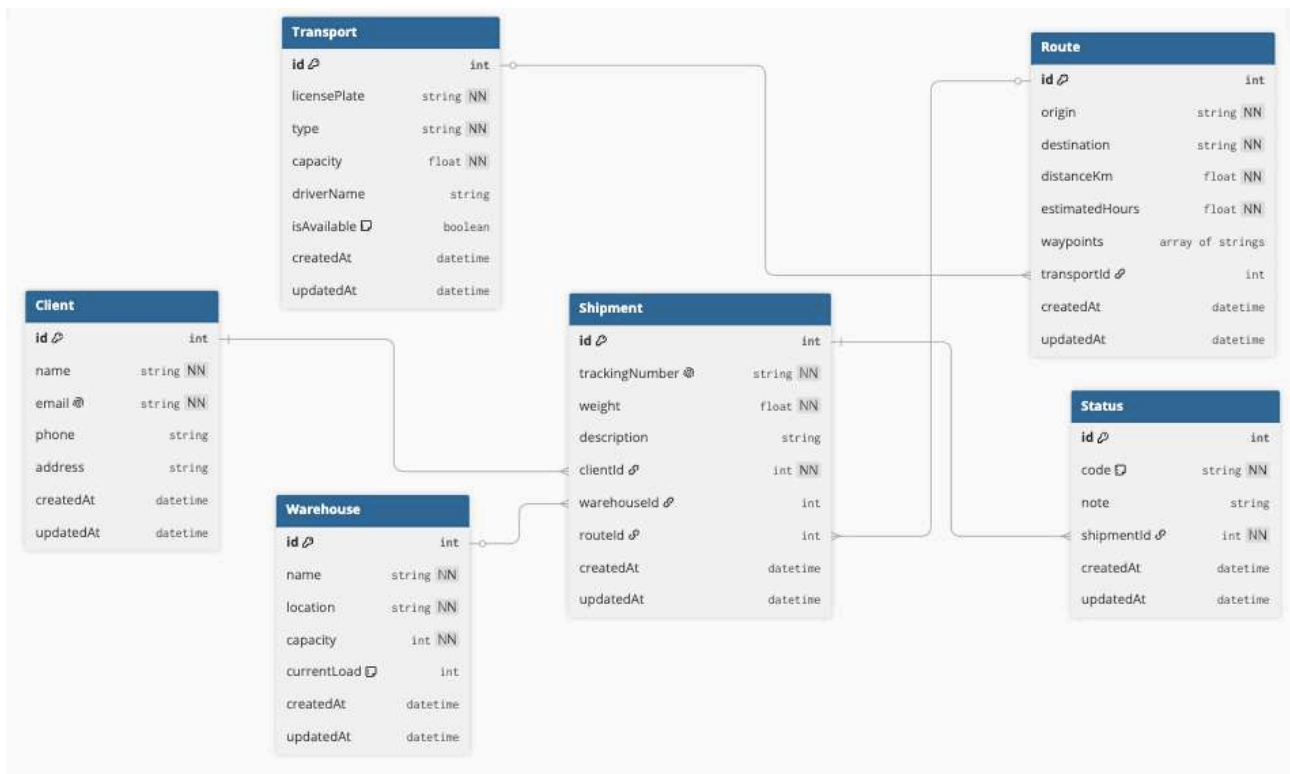


Рис. 1. ER-діаграма

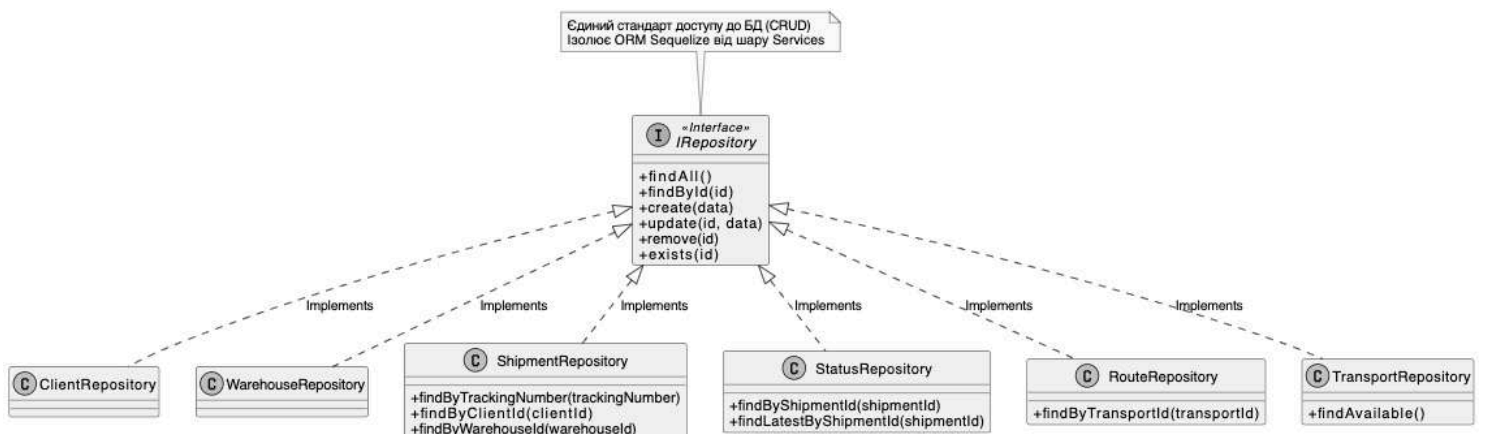


Рис. 2. Repository layer

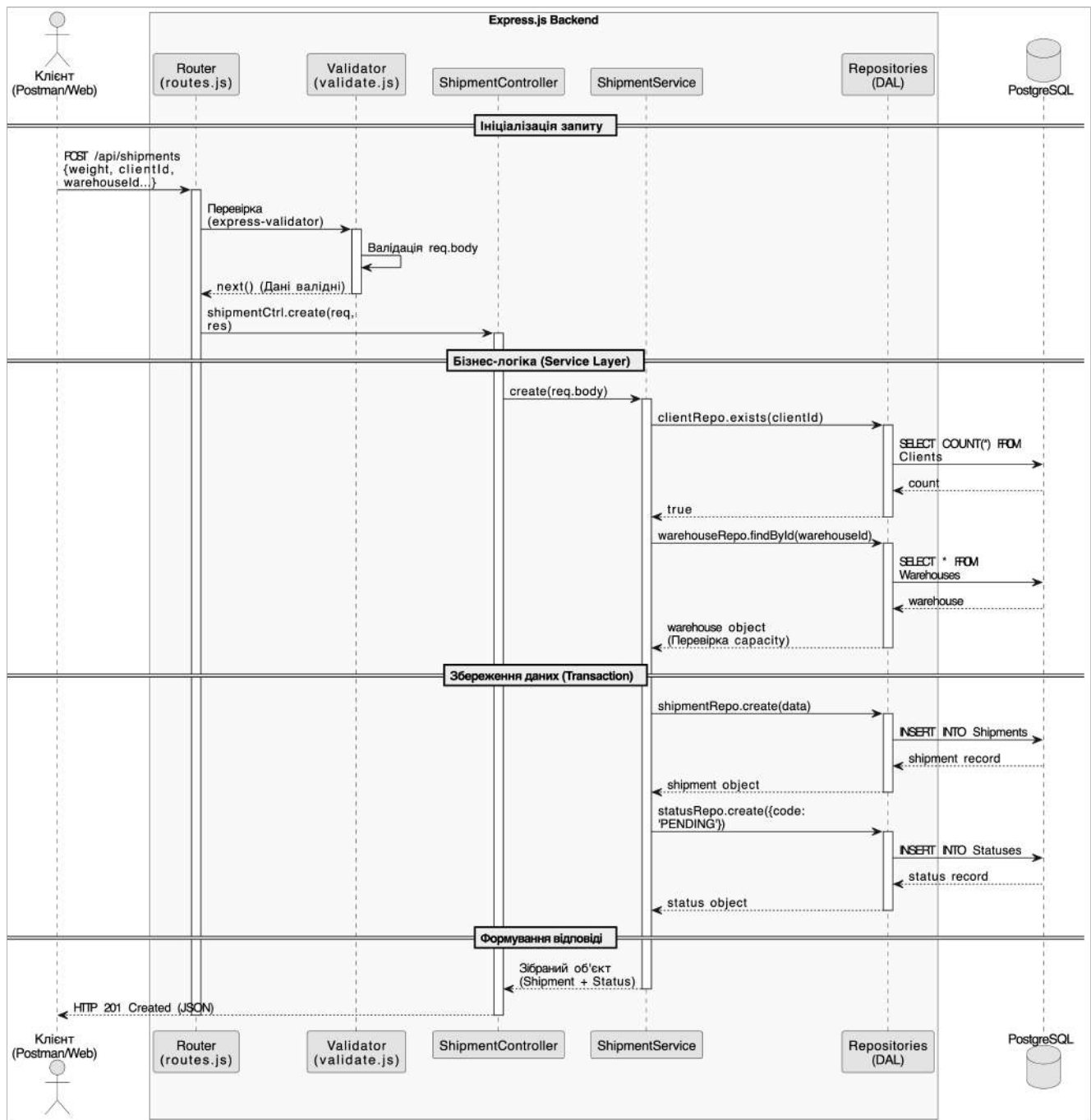


Рис. 3. Архітектурна схема взаємодії

2. Реалізація:

Я обрав базу даних PostgreSQL

Приклад оновленої моделі Client.js:

```

const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Client = sequelize.define('Client', {
  id: { type: DataTypes.INTEGER, primaryKey: true, autoIncrement: true },
  name: { type: DataTypes.STRING, allowNull: false },
  email: { type: DataTypes.STRING, unique: true, allowNull: false },
  phone: { type: DataTypes.STRING, allowNull: true },
  address: { type: DataTypes.STRING, allowNull: true }
});

module.exports = Client;
  
```

Приклад оновленого контролера clientController.js:

```
const service = require('../services/clientService');

module.exports = {
  getAll: async (req, res, next) => {
    try {
      res.json(await service.getAll());
    } catch (e) { next(e); }
  },

  getById: async (req, res, next) => {
    try {
      res.json(await service.getById(req.params.id));
    } catch (e) { next(e); }
  },

  create: async (req, res, next) => {
    try {
      res.status(201).json(await service.create(req.body));
    } catch (e) { next(e); }
  },

  update: async (req, res, next) => {
    try {
      res.json(await service.update(req.params.id, req.body));
    } catch (e) { next(e); }
  },

  remove: async (req, res, next) => {
    try {
      await service.remove(req.params.id);
      res.status(204).send();
    } catch (e) { next(e); }
  }
};
```

Приклад оновленого сервіса clientService.js:

```
const repo = require('../repositories/clientRepository');
const shipmentRepo = require('../repositories/shipmentRepository');

module.exports = {
  getAll: async () => await repo.findAll(),

  getById: async (id) => {
    const client = await repo.findById(id);
    if (!client) throw { status: 404, message: `Client with id=${id} not found` };
    return client;
  },

  create: async (data) => {
    const clients = await repo.findAll();
    const duplicate = clients.find(c => c.email === data.email);
    if (duplicate) throw { status: 409, message: `Email "${data.email}" is already in use` };
    return await repo.create(data);
  },

  update: async (id, data) => {
    if (!(await repo.exists(id))) throw { status: 404, message: `Client with id=${id} not found` };
  }
};
```

```

    if (data.email) {
      const clients = await repo.findAll();
      const duplicate = clients.find(c => c.email === data.email && c.id !==
Number(id));
      if (duplicate) throw { status: 409, message: `Email "${data.email}" is
already in use` };
    }

    return await repo.update(id, data);
  },

  remove: async (id) => {
    if (!(await repo.exists(id))) throw { status: 404, message: `Client with
id=${id} not found` };

    const shipments = await shipmentRepo.findByClientId(id);
    if (shipments.length > 0) {
      throw { status: 409, message: `Cannot delete client id=${id}: has active
shipments` };
    }

    await repo.remove(id);
  },
};

```

Приклад оновленого репозиторія clientRepository.js:

```

const { Client } = require('../models');

module.exports = {
  findAll: async () => await Client.findAll(),

  findById: async (id) => await Client.findById(id),

  create: async (data) => await Client.create(data),

  update: async (id, data) => {
    await Client.update(data, { where: { id } });
    return await Client.findById(id);
  },

  remove: async (id) => {
    const deletedCount = await Client.destroy({ where: { id } });
    return deletedCount > 0;
  },

  exists: async (id) => {
    const count = await Client.count({ where: { id } });
    return count > 0;
  },
};

```

Файл конфігурації бд:

```

const { Sequelize } = require('sequelize');
require('dotenv').config();

const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  {
    host: process.env.DB_HOST,
    dialect: 'postgres',
    logging: false,
  }
);

```

```
}  
);  
  
module.exports = sequelize;
```

3. Тестування:

Повний CRUD (Успішний сценарій на прикладі Клієнта)

1.Create:

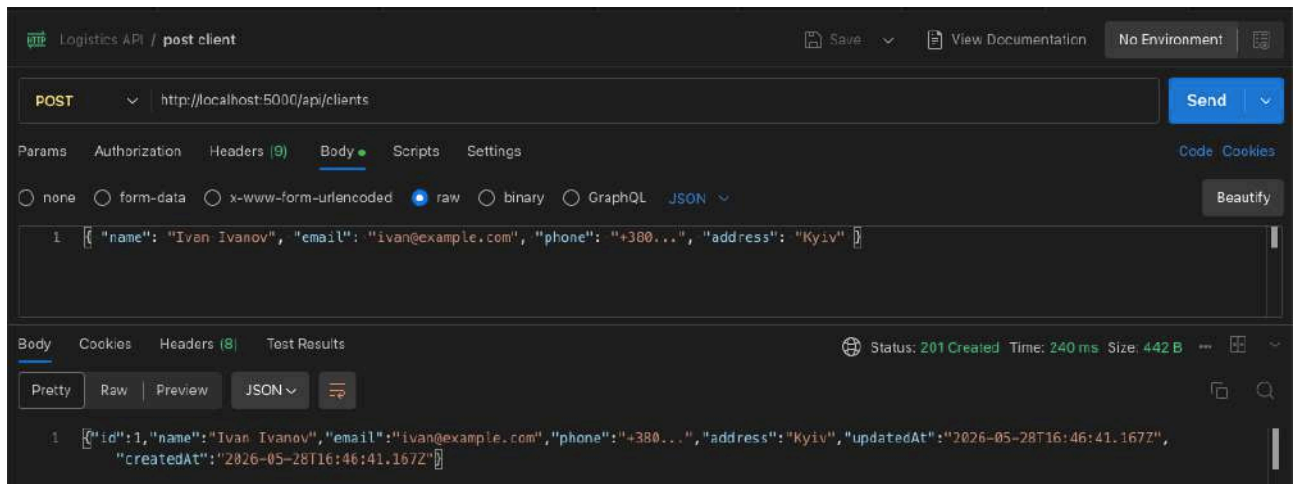


Рис. 1. Успішне створення клієнта. Виконання POST /api/clients

2.Read:

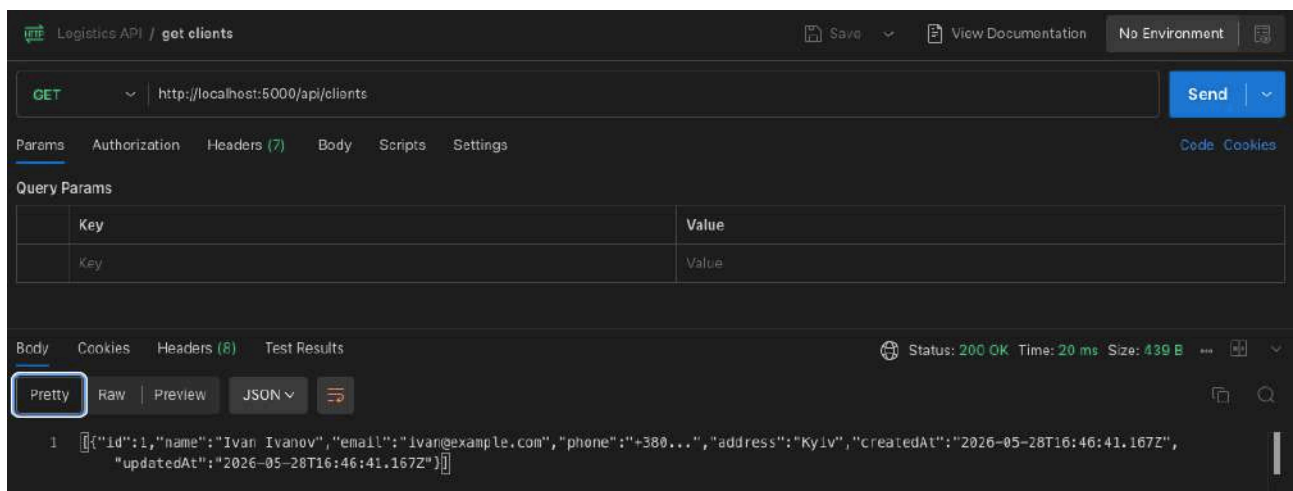


Рис. 2. Успішне виведення клієнта. Виконання GET /api/clients

3.Update:

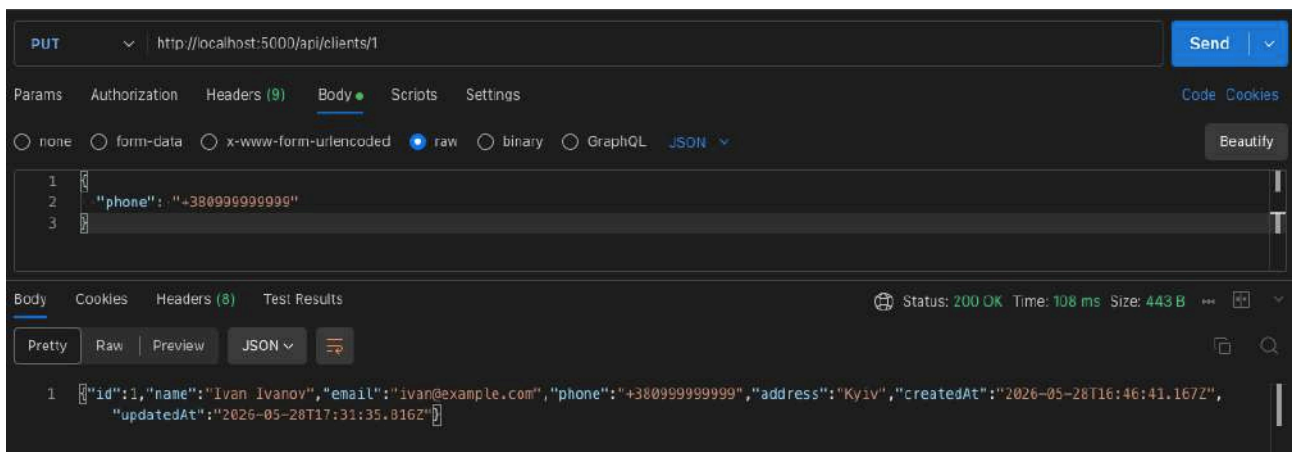


Рис. 3. Успішне редагування даних клієнта. Виконання PUT /api/clients/:id

4.Delete:

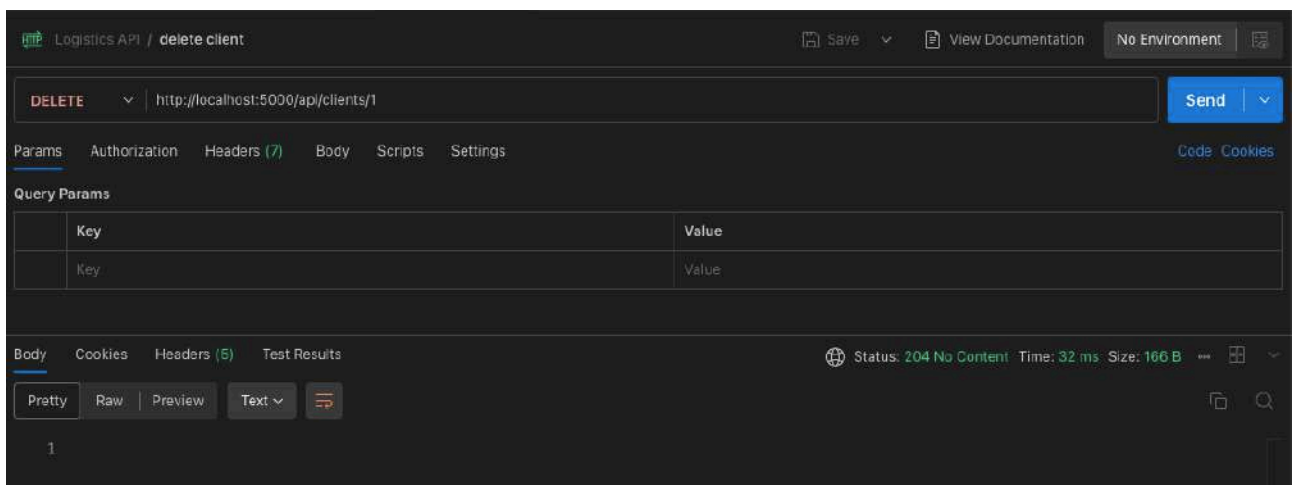


Рис. 4. Успішне видалення даних клієнта. Виконання DELETE /api/clients/:id

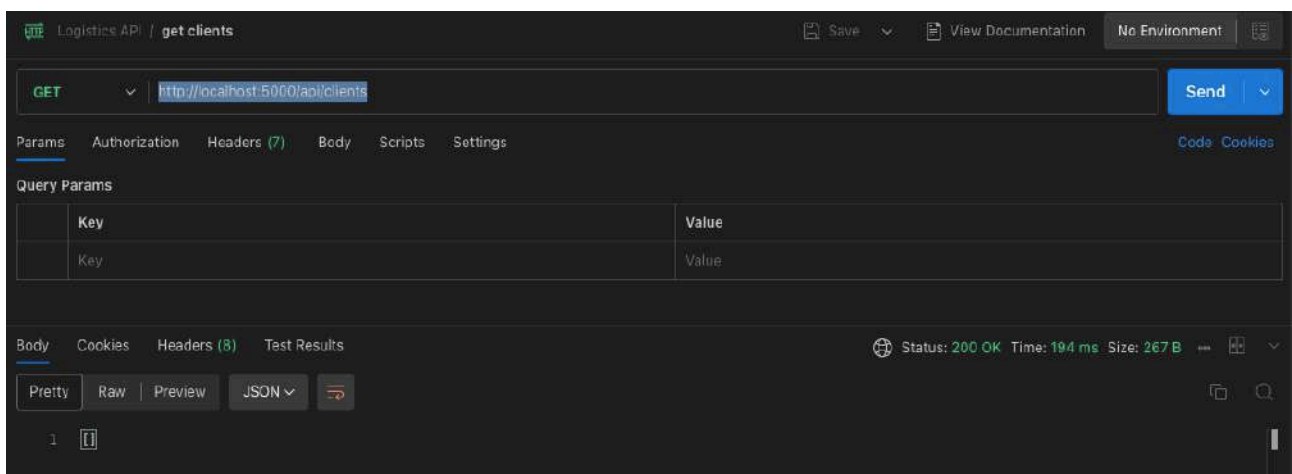


Рис. 5. Підтвердження, що дані видалені. Виконання GET /api/clients

Перевірка некоректних сценаріїв

1. Помилка валідації:

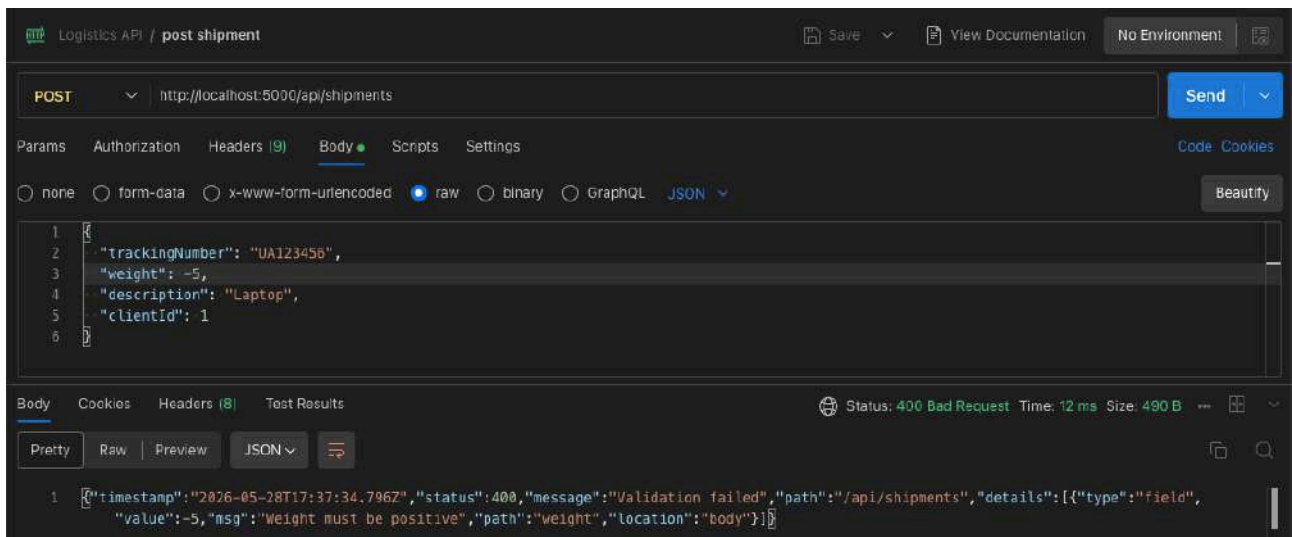


Рис. 6. 400 Bad Request. Помилка через від'ємну вагу.

2. Об'єкт не знайдено:

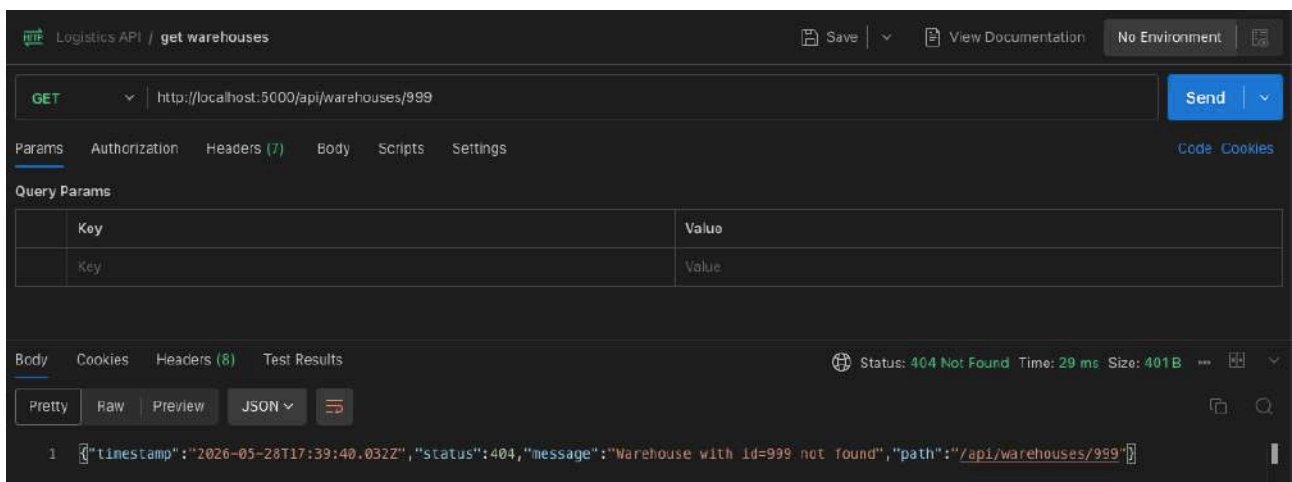


Рис. 7. 404 Not Found. Помилка через невірне id об'єкта.

3. Бізнес-конфлікт:

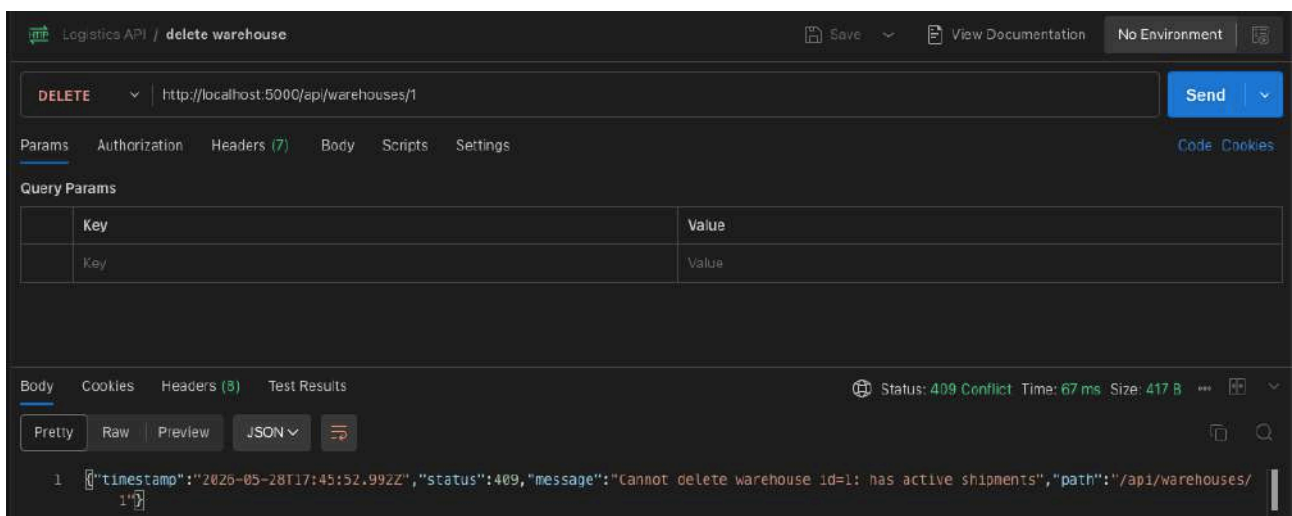


Рис. 8. 409 Conflict. Заблоковується видалення складу, на якому зареєстрована посилка

Перевірка транзакційності

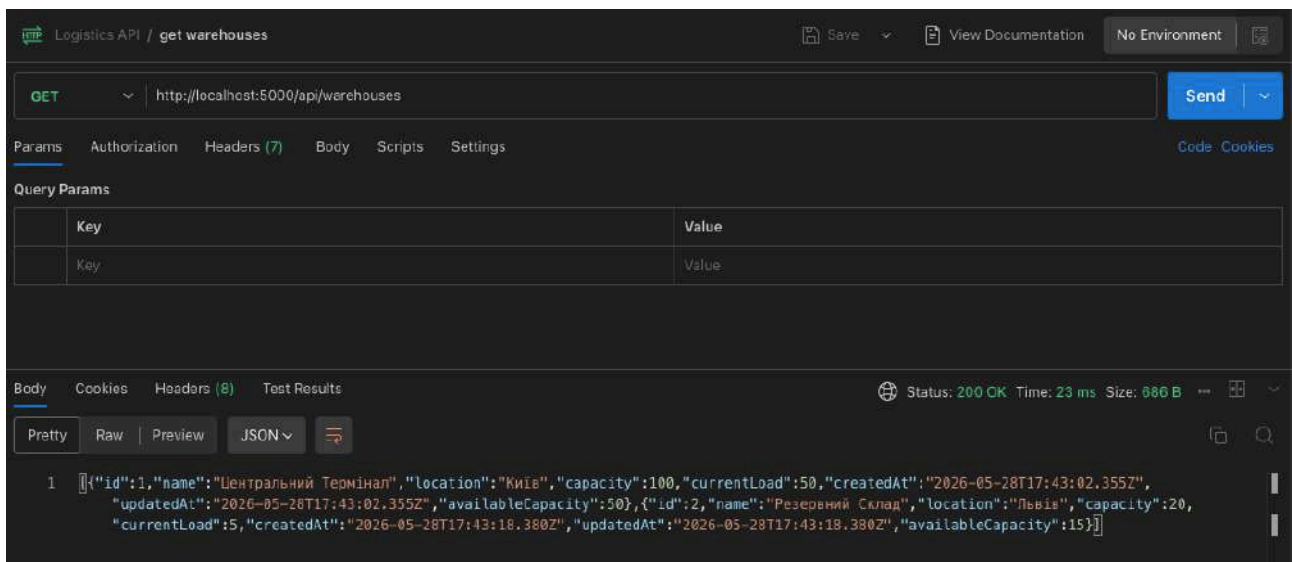


Рис. 9. Створені два склади, з посилкою на першому складі.

На місці запиту переміщення посилки між складами було штучно імітовано помилку для демонстрації транзакційності.

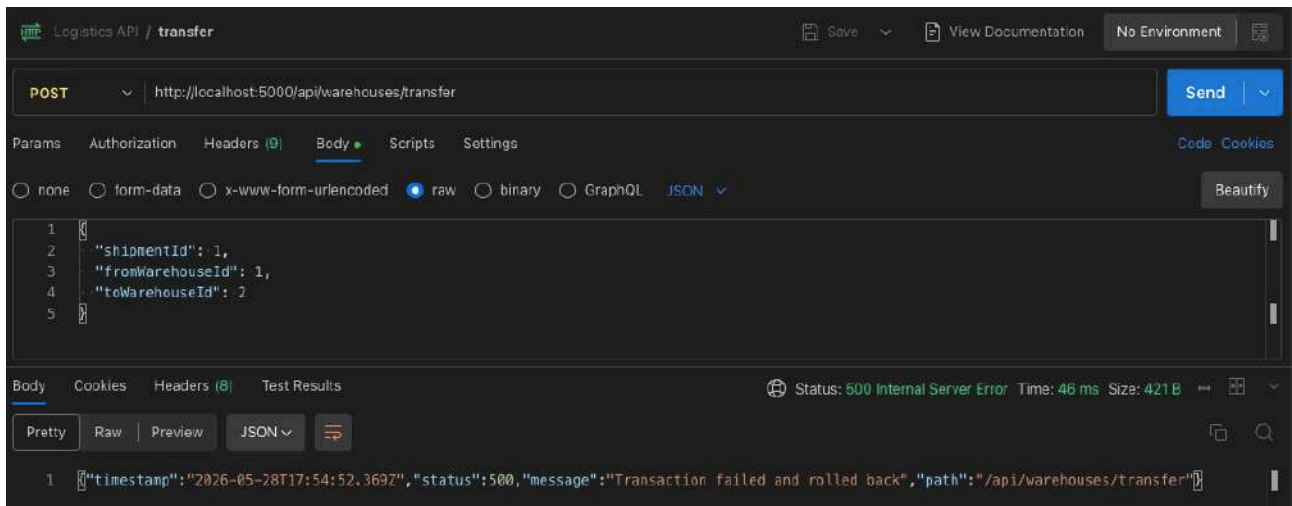


Рис. 10. Помилка переміщення.

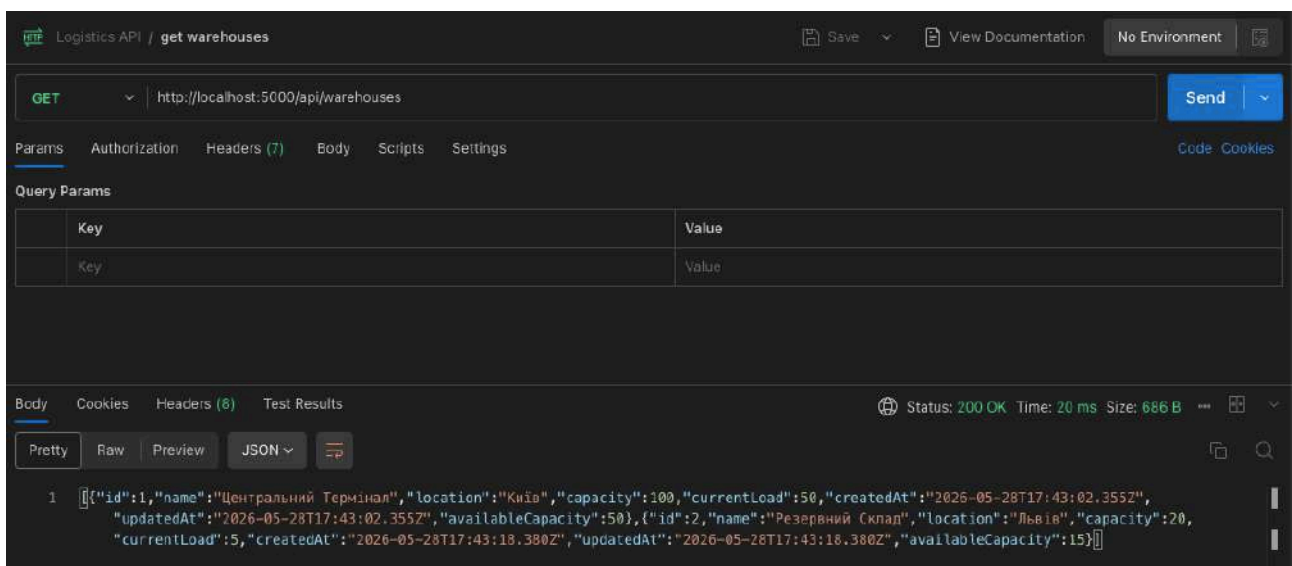


Рис. 11. Місце на складах не змінилося

4. Порівняльний аналіз:

In-memory vs база даних

Зберігання даних в оперативній пам'яті (in-memory) є зручним на етапі прототипування, але інформація безповоротно втрачається при кожному перезапуску сервера, а масштабування обмежується обсягом RAM. Натомість реляційна база даних PostgreSQL забезпечує надійне збереження інформації на жорсткому диску та гарантує консистентність даних. Замість ручного пошуку по масивах, СУБД дозволяє швидко знаходити потрібні записи серед мільйонів рядків завдяки індексації, а також автоматично контролює унікальність полів і правильність форматів на рівні таблиць.

Зміни в архітектурі

Перехід на реальну базу даних вимагав впровадження асинхронного програмування, тому всі методи репозиторіїв, сервісів та контролерів були переписані з використанням конструкцій `async/await`. Завдяки попередньо закладеній багатошаровій архітектурі бізнес-логіка системи залишилася повністю недоторканою. Усі зміни локалізувалися виключно в Data Access Layer (репозиторіях), де операції над масивами були замінені на виклики методів ORM Sequelize, яка взяла на себе генерацію SQL-запитів.

Переваги персистентності

Головною перевагою є незалежність життєвого циклу даних від стану Node.js сервера, що дозволяє безпечно оновлювати чи перезавантажувати систему. Персистентність забезпечує можливість створення резервних копій (дампів) та проведення складної аналітики. Крім того, перенесення відповідальності за зв'язки між сутностями на рівень бази даних (Foreign Keys) гарантує строгу цілісність системи, унеможливаючи ситуації, коли видаляється склад, на якому фізично ще зареєстровані посилки.

Висновок:

У ході виконання роботи було успішно спроектовано та реалізовано RESTful API для логістичної системи з використанням технологій Node.js та Express. Головним досягненням став перехід від тимчасового зберігання даних в оперативній пам'яті до надійної персистентної бази даних PostgreSQL за допомогою ORM Sequelize. Завдяки застосуванню багатошарової архітектури з чітким розділенням на контролери, сервіси та репозиторії, вдалося створити гнучкий та масштабований програмний код, де бізнес-логіка повністю ізольована від механізмів доступу до даних. Впровадження глобальної обробки винятків, строгих правил валідації та механізму транзакцій для складних операцій

забезпечило високу надійність системи та захист від некоректних дій. У результаті розроблено повноцінний, відмовостійкий бекенд, який гарантує строгу консистентність даних і є повністю готовим до подальшого розвитку або інтеграції з клієнтськими інтерфейсами.